Project 4: Advanced Tracking

Katarina Brešar

I. Introduction

In this assignment, we tested how motion models combined with the Kalman filter can be used to predict the future position of an object. We tested three different models: Random Walk, Nearly Constant Velocity and Nearly Constant Acceleration and analyzed how the parameters of the Kalman filter affect the performance of the tracking. We also implemented a Particle filter tracker... TODO

In the second part, we implemented a Particle filter tracker that utilizes the NCV motion model and a color histogram visual model. We integrated our tracker into the VOT-lite evaluation toolkit and analyzed its performance across different parameters, motion models, and sample sizes on the VOT2014 dataset.

II. Experiments

A. Motion Models and Kalman Filters

We first ran the Kalman filter method on a spiral curve using RW, NCV and NCA motion models. The matrices that we used are in the appendix section. Using different values of parameters q and r we observed how the filters performance changed. In figure 1 we can see the effect of setting $q = [1, 5, 100]$ and $r = [1, 5, 100]$. We're interested in the relationship between these two parameters. When they are both small the filter is trusting both the model and the measurments. Setting q to 100 (and leaving r small) meant that the motion model is highly uncertain, so the object might deviate a lot from the predicted path. So the filter trusts the measurments almost entirely (as we can see in the figure). Setting q to a smaller value the filter will trust the motion model more so the trajectory is smoother, but it doesn't follow the measurments as closely. Setting r to a smaller value means that the measurments are more accurate, so the filter will trust them more. Like in the figure where q was large and r was small for all three models. But when r is large the filter will rely mostly on the internal prediction of the model. As seen in the last column of the figure. When q is small and $r = 100$ the filter is not following the measurments at all. Especially in random walk we can see how the filter is lagging behind the measurments at sharp turns as it assumes the object is static. Which is why it's also mostly always "behind" the measurments with other parameters as well. For the NCV model it follows the measured spiral better, because it also tracks velocity. The NCA model overshot the curve at tight corners because it also tracks acceleration, which becomes a problem when the curves radius changed quickly.

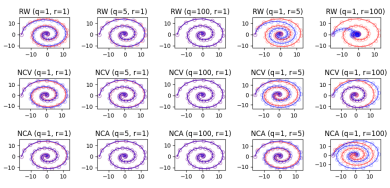


Figure 1. Kalman filter performance. Blue curve: filtered trajectory, Red curve: measured trajectory

We also ran the tests on a rectangle path as seen in figure 2. We can see that all the models struggle with the 90 degree corners when $q = 1, r = 5$ (meaning the filter trusts the model more than the measurments) since the assume the objects travels at a straight line. The RW is the worst at handling the corners, because it has no momentum, the other two are better at it, but still overshoot the corners. When q is large and r is small, random walk fails completely (as expected with no momentum) and NCA performs the best as it can quickly adjust to the new direction.

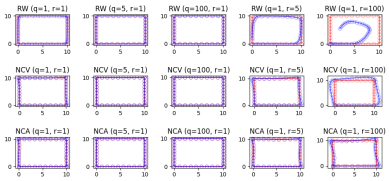


Figure 2. Kalman filter performance on rectangle path. Blue curve: filtered trajectory, Red curve: measured trajectory

We also tested the filters on a sine wave path as seen in figure 3. Again the RW model performs the worst when it the filter trusts the model more than the measurments. While the NCV model performs well, it's faster than the measurments since it assumes a nearly constant velocity. But the NCA model performs the best, because it quickly adjusts to the changing direction.

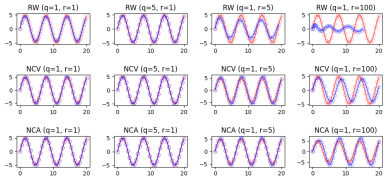


Figure 3. Kalman filter performance on sine wave path. Blue curve: filtered trajectory, Red curve: measured trajectory

B. Particle Filter Tracker

The Particle filter was implemented by maintaining N particles representing possible states of the target. At each frame, we resampled the particles based on their weights, propagated them using the selected motion model with added Gaussian noise (sampled from Q), and recalculated their weights using the Hellinger distance between the extracted color histogram of the particle patch and the target's visual model. The final bounding box was determined by a weighted average of the particles.We used an Epanechnikov kernel for histogram extraction to give more importance to the center of the patch, and updated the target visual model at each frame using an exponential moving average ($\alpha = 0.05$).

1) Baseline Performance: Using the NCV motion model, $N = 100$ particles, and an HSV color space, we tested the tracker on 5 sequences from the VOT dataset. The parameter q was set proportionally to the target size ($q = 5.0$).

The tracker proved robust on sequences with consistent target color and smooth motion (e.g., car, jogging). However, in sequences with rapid scale changes or severe illumination shifts (e.g., hand), the color histogram visual model struggled, resulting in lower overlap and more failures. The average speed was approximately 69 FPS, which is computationally heavier than the correlation filter from Exercise 3 due to the multiple patch extractions required per frame.

C. Analysis of Tracker Performance

Motion Models within the Tracker: We compared the tracker's accuracy and robustness by swapping the underlying motion models (Table II).

NCV significantly outperformed RW and NCA. RW failed when targets moved rapidly, as the particles only spread via noise and did not carry momentum, causing the target to leave the particle spread. NCA performed worse than NCV because the acceleration state variables became highly unstable given the noise in bounding box updates, scattering the particles too widely.The parameter q dictates the spread of the particles. We found it must be proportional to the target size; if q is too small, particles collapse into a tight cluster and cannot recover if the target makes sudden movements. If q is too large, particles spread across the entire image, capturing background noise and causing catastrophic drifting.Number of Particles (N): We observed how the number of samples impacted performance.

Using too few particles ($N = 20$) resulted in poor state representation, causing frequent failures. Increasing N to 100 drastically improved accuracy and robustness. However, increasing N beyond 100 yielded diminishing returns in overlap while causing tracking speed to plummet linearly, as histogram extraction is required for every particle.

III. Conclusion

In this exercise, we implemented advanced tracking utilizing Kalman and Particle filters. We found that the Nearly-Constant Velocity (NCV) model provides the best motion prior for standard visual objects, balancing momentum and stability. Furthermore, optimizing the number of particles is crucial; $N = 100$ served as the ideal "sweet spot" providing robust tracking at near real-time speeds (69 FPS). While the color histogram is a reliable visual model, it remains susceptible to background distractors of similar colors, suggesting that combining it with a spatial template could further improve robustness.

Appendix

1. Random walk: State vector: $X = [x, y]^\top$

$$F = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix},$$
$$\Phi = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$
$$L = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$
$$Q = q\Delta t \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$
$$H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

2. NCV: State vector: $X = [x, y, \dot{x}, \dot{y}]^\top$

$$F = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$
$$L = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix},$$
$$\Phi = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$
$$Q = q \begin{bmatrix} \frac{\Delta t^3}{3} & 0 & \frac{\Delta t^2}{2} & 0 \\ 0 & \frac{\Delta t^3}{3} & 0 & \frac{\Delta t^2}{2} \\ \frac{\Delta t^2}{2} & 0 & \Delta t & 0 \\ 0 & \frac{\Delta t^2}{2} & 0 & \Delta t \end{bmatrix},$$
$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

3. NCA: State vector: $X = [x, y, \dot{x}, \dot{y}, \ddot{x}, \ddot{y}]^\top$

$$F = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix},$$
$$\Phi = \begin{bmatrix} 1 & 0 & \Delta t & 0 & \frac{\Delta t^2}{2} & 0 \\ 0 & 1 & 0 & \Delta t & 0 & \frac{\Delta t^2}{2} \\ 0 & 0 & 1 & 0 & \Delta t & 0 \\ 0 & 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix},$$
$$L = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix},$$
$$Q = q \begin{bmatrix} \frac{\Delta t^5}{20} & 0 & \frac{\Delta t^4}{8} & 0 & \frac{\Delta t^3}{6} & 0 \\ 0 & \frac{\Delta t^5}{20} & 0 & \frac{\Delta t^4}{8} & 0 & \frac{\Delta t^3}{6} \\ \frac{\Delta t^4}{8} & 0 & \frac{\Delta t^3}{3} & 0 & \frac{\Delta t^2}{2} & 0 \\ 0 & \frac{\Delta t^4}{8} & 0 & \frac{\Delta t^3}{3} & 0 & \frac{\Delta t^2}{2} \\ \frac{\Delta t^3}{6} & 0 & \frac{\Delta t^2}{2} & 0 & \Delta t & 0 \\ 0 & \frac{\Delta t^3}{6} & 0 & \frac{\Delta t^2}{2} & 0 & \Delta t \end{bmatrix},$$
$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$